PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING ASSIGNMENT

Name	M. Sahithi
Roll Number	26000951
Branch	CSM
Section	2

Selected 10 case studies with problem statements, objectives, assumptions, algorithms, pseudocode, Python source code, test cases, sample outputs, and conclusions.

Course: Python Programming

Contents

1. University Scholarship Allocation
2. Progressive Electricity Billing
3. Hospital Billing and Patient Classification
4. Employee Payroll and Deduction Analysis
5. Multi-Plan Telecommunications Cost Optimization
6. Railway Fare and Reservation Calculator
7. Retail Billing with Multiple Discount Rules
8. Manufacturing Cost and Profitability Analysis
9. Multi-Currency Travel Budget Analysis
10. Campus Event Financial Planning

1. University Scholarship Allocation

Problem Statement

Determine which applicants qualify for a scholarship using aggregate marks, subject minimums, attendance, and family income. Rank eligible applicants by marks.

Objectives

• Check all eligibility conditions.

• Calculate aggregate percentage and shortlist applicants.

• Rank eligible applicants by percentage, then attendance.

Assumptions

• Each applicant record contains name, marks in three subjects, attendance percentage, and annual family income.

• Eligibility: each subject mark is at least 40, aggregate percentage at least 75%, attendance at least 85%, and income at most ■250,000.

• Ties are broken by higher attendance.

Algorithm

1. Start.

2. Read the applicant records.

3. Calculate total marks and percentage for each applicant.

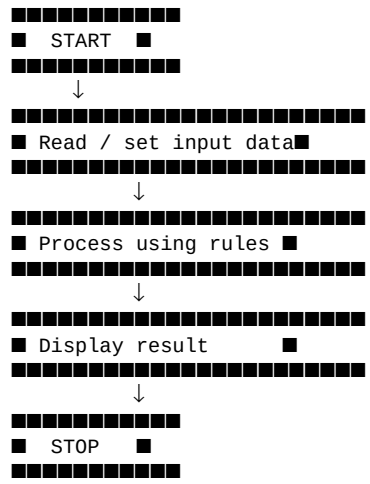
4. Check subject marks, percentage, attendance, and income conditions.

5. Store eligible applicants with percentage and attendance.

6. Sort by percentage descending, then attendance descending.

7. Display the eligible ranking and stop.

Flowchart



Pseudocode

```
START
FOR each applicant
  total = sum(subject marks)
  percentage = total / 3
  IF every subject >= 40 AND percentage >= 75
     AND attendance >= 85 AND income <= 250000
       add applicant to eligible list
SORT eligible by percentage descending, attendance descending
DISPLAY eligible list
STOP
```

Python Source Code

```
applicants = [
    ("Asha", 88, 79, 91, 92, 180000),
    ("Bharat", 75, 80, 78, 88, 220000),
    ("Charan", 92, 35, 90, 95, 150000),
    ("Divya", 81, 84, 86, 90, 240000),
]

eligible = []
for name, s1, s2, s3, attendance, income in applicants:
    percentage = (s1 + s2 + s3) / 3
    if (min(s1, s2, s3) >= 40 and percentage >= 75
            and attendance >= 85 and income <= 250000):
        eligible.append((name, percentage, attendance))

eligible.sort(key=lambda item: (item[1], item[2]), reverse=True)
print("Eligible scholarship applicants:")
for rank, (name, percentage, attendance) in enumerate(eligible, 1):
    print(f"{rank}. {name}: {percentage:.2f}%, attendance {attendance}%")
```

Test Cases

Input / Test	Working / Expected Result
Asha	88,79,91; attendance 92; income 180000 Expected: Eligible; 86.00%
Bharat	75,80,78; attendance 88; income 220000 Expected: Eligible; 77.67%
Charan	92,35,90; attendance 95; income 150000 Expected: Rejected: one subject below 40
Divya	81,84,86; attendance 90; income 240000 Expected: Eligible; 83.67%

Sample Output

```
Eligible scholarship applicants:
1. Asha: 86.00%, attendance 92%
2. Divya: 83.67%, attendance 90%
3. Bharat: 77.67%, attendance 88%
```

Conclusion

The program applies the stated conditions and calculations to the sample data and produces the expected result. The case study demonstrates the use of variables, operators, conditional statements, loops, lists, and tuples as appropriate.

2. Progressive Electricity Billing

Problem Statement

Calculate electricity bills for households using consumption slabs, a fixed charge, and a surcharge when consumption is high.

Objectives

• Apply slab-wise rates correctly.

• Add fixed charges and any surcharge.

• Compare bills for different monthly readings.

Assumptions

• First 100 units cost ■2/unit; next 100 units cost ■3/unit; units above 200 cost ■5/unit.

• Fixed charge is ■50. Consumption above 250 units receives a 5% surcharge on energy charges.

Algorithm

1. Read each household's consumption.

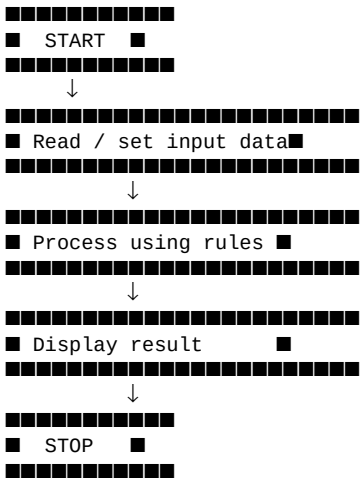
2. Calculate energy charge using progressive slabs.

3. Add fixed charge.

4. Apply 5% surcharge if units exceed 250.

5. Display the final bill.

Flowchart



Pseudocode

```
START
FOR units in monthly_readings
  IF units <= 100: energy = units * 2
  ELSE IF units <= 200: energy = 100*2 + (units-100)*3
  ELSE: energy = 100*2 + 100*3 + (units-200)*5
  surcharge = energy*0.05 if units > 250 else 0
  bill = energy + 50 + surcharge
  DISPLAY units, bill
STOP
```

Python Source Code

```
def calculate_bill(units):
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
```

```
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5

    surcharge = energy * 0.05 if units > 250 else 0
    return energy + 50 + surcharge

monthly_readings = [80, 150, 230, 300]
for month, units in enumerate(monthly_readings, 1):
    print(f"Month {month}: {units} units -> Rs. {calculate_bill(units):.2f}")
```

Test Cases

Input / Test	Working / Expected Result
80 units	Energy 160 + fixed 50 Expected: ■210.00
150 units	Energy 200 + 150 + fixed 50 Expected: ■400.00
230 units	Energy 200 + 300 + 150 + fixed 50 Expected: ■700.00
300 units	Energy 200 + 300 + 500; 5% surcharge 50; fixed 50 Expected: ■1,100.00

Sample Output

```
Month 1: 80 units -> Rs. 210.00
Month 2: 150 units -> Rs. 400.00
Month 3: 230 units -> Rs. 700.00
Month 4: 300 units -> Rs. 1100.00
```

Conclusion

The program applies the stated conditions and calculations to the sample data and produces the expected result. The case study demonstrates the use of variables, operators, conditional statements, loops, lists, and tuples as appropriate.

3. Hospital Billing and Patient Classification

Problem Statement

Calculate a patient's hospital bill from consultation, diagnostic, and treatment charges, then apply a concession when the policy conditions are met.

Objectives

• Add the listed medical service charges.

• Apply a concession conditionally.

• Classify the final bill using a payment threshold.

Assumptions

• A 10% concession applies when the patient is a senior citizen (age 60 or above).

• A final bill above ■5,000 is classified as High; otherwise it is Standard.

Algorithm

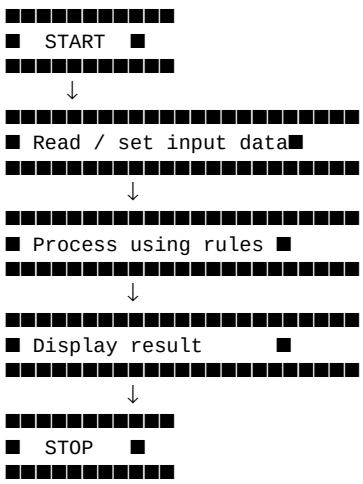
1. Read age and service charges.

2. Add consultation, diagnostic, and treatment charges.

3. If age is at least 60, subtract 10% concession.

4. Classify the final bill and display details.

Flowchart



Pseudocode

```
START
subtotal = consultation + diagnostics + treatment
IF age >= 60: discount = subtotal * 0.10
ELSE: discount = 0
final_bill = subtotal - discount
IF final_bill > 5000: category = High
ELSE: category = Standard
DISPLAY final_bill, category
STOP
```

Python Source Code

```
def hospital_bill(age, consultation, diagnostics, treatment):
    subtotal = consultation + diagnostics + treatment
    discount = subtotal * 0.10 if age >= 60 else 0
    final_bill = subtotal - discount
    category = "High" if final_bill > 5000 else "Standard"
    return subtotal, discount, final_bill, category

records = [
```

```
    ("Patient 1", 65, 500, 1800, 4000),
    ("Patient 2", 25, 400, 900, 2200),
]
for name, age, consultation, diagnostics, treatment in records:
    subtotal, discount, final_bill, category = hospital_bill(
        age, consultation, diagnostics, treatment)
    print(f"{name}: subtotal Rs.{subtotal:.2f}, concession Rs.{discount:.2f}, "
          f"final Rs.{final_bill:.2f}, category {category}")
```

Test Cases

Input / Test	Working / Expected Result
Age 65; charges 500, 1800, 4000	Subtotal ■6,300; 10% concession ■630 Expected: ■5,670; High
Age 25; charges 400, 900, 2200	Subtotal ■3,500; no concession Expected: ■3,500; Standard

Sample Output

```
Patient 1: subtotal Rs.6300.00, concession Rs.630.00, final Rs.5670.00, category High
Patient 2: subtotal Rs.3500.00, concession Rs.0.00, final Rs.3500.00, category Standard
```

Conclusion

The program applies the stated conditions and calculations to the sample data and produces the expected result. The case study demonstrates the use of variables, operators, conditional statements, loops, lists, and tuples as appropriate.

4. Employee Payroll and Deduction Analysis

Problem Statement

Calculate an employee's gross salary from basic pay, allowances, overtime, and incentives, then calculate a simple progressive deduction.

Objectives

• Compute gross salary components.

• Apply the specified deduction slabs.

• Print a salary statement.

Assumptions

• Gross salary = basic + allowances + overtime + incentive.

• Deduction is 5% of the first ■20,000 and 10% of the amount above ■20,000.

Algorithm

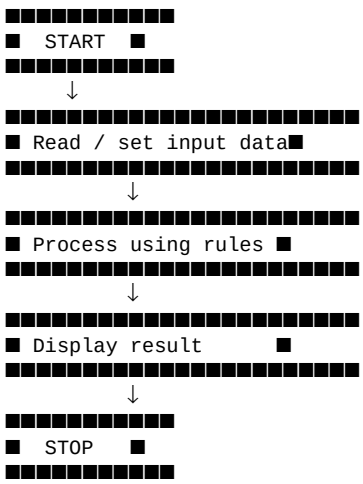
1. Read salary components.

2. Calculate gross salary.

3. Calculate deduction using the two slabs.

4. Calculate net salary and print the statement.

Flowchart



Pseudocode

```
START
gross = basic + allowances + overtime + incentive
IF gross <= 20000: deduction = gross * 0.05
ELSE: deduction = 20000*0.05 + (gross-20000)*0.10
net = gross - deduction
DISPLAY gross, deduction, net
STOP
```

Python Source Code

```
def payroll(basic, allowances, overtime, incentive):
    gross = basic + allowances + overtime + incentive
    if gross <= 20000:
        deduction = gross * 0.05
    else:
        deduction = 20000 * 0.05 + (gross - 20000) * 0.10
    return gross, deduction, gross - deduction

employees = [
    ("Employee A", 18000, 2500, 1000, 500),
```

```
    ("Employee B", 15000, 2000, 0, 1000),
]
for name, basic, allowances, overtime, incentive in employees:
    gross, deduction, net = payroll(basic, allowances, overtime, incentive)
    print(f"{name}: gross Rs.{gross:.2f}, deduction Rs.{deduction:.2f}, "
          f"net Rs.{net:.2f}")
```

Test Cases

Input / Test	Working / Expected Result
Basic 18000, allowances 2500, overtime 1000, incentive 500	Gross ■22,000; deduction ■1,000 + ■200 Expected: Net ■20,800
Basic 15000, allowances 2000, overtime 0, incentive 1000	Gross ■18,000; deduction 5% Expected: Net ■17,100

Sample Output

```
Employee A: gross Rs.22000.00, deduction Rs.1200.00, net Rs.20800.00
Employee B: gross Rs.18000.00, deduction Rs.900.00, net Rs.17100.00
```

Conclusion

The program applies the stated conditions and calculations to the sample data and produces the expected result. The case study demonstrates the use of variables, operators, conditional statements, loops, lists, and tuples as appropriate.

5. Multi-Plan Telecommunications Cost Optimization

Problem Statement

Compare mobile plans using monthly data usage and choose the lowest-cost plan that fits the user's budget.

Objectives

• Calculate total cost for each plan.

• Reject plans that exceed the budget.

• Choose the least expensive eligible plan.

Assumptions

• Plan A: ■199 includes 2 GB; excess is ■30/GB.

• Plan B: ■299 includes 5 GB; excess is ■20/GB.

• Plan C: ■399 includes 10 GB; no extra charge up to 10 GB, then ■25/GB.

• Partial excess GB is charged proportionally; budget is ■350.

Algorithm

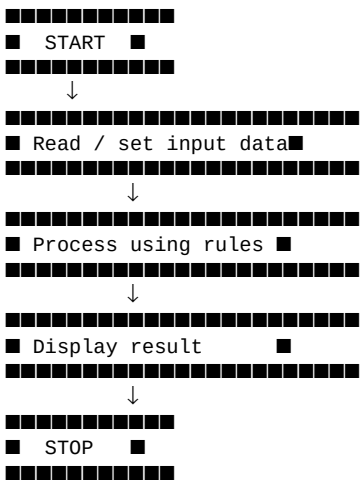
1. Read data usage and budget.

2. Calculate each plan's base plus excess cost.

3. Keep plans within budget.

4. Select the lowest-cost eligible plan or report none.

Flowchart



Pseudocode

```
START
plans = (base, included_data, excess_rate)
FOR each plan
  extra = max(0, usage - included_data)
  cost = base + extra * excess_rate
  IF cost <= budget: store plan and cost
IF eligible plans exist: choose minimum cost
ELSE: report no suitable plan
STOP
```

Python Source Code

```
plans = [
    ("Plan A", 199, 2, 30),
    ("Plan B", 299, 5, 20),
    ("Plan C", 399, 10, 25),
]
usage_gb = 6
```

```
budget = 350
eligible = []

for name, base, included, rate in plans:
    extra_cost = max(0, usage_gb - included) * rate
    total = base + extra_cost
    print(f"{name}: Rs.{total:.2f}")
    if total <= budget:
        eligible.append((total, name))

if eligible:
    cost, name = min(eligible)
    print(f"Best plan within budget: {name} at Rs.{cost:.2f}")
else:
    print("No plan fits the budget.")
```

Test Cases

Input / Test	Working / Expected Result
Usage 6 GB; budget ■350	A=199+(4×30); B=299+(1×20); C=399 Expected: A ■319, B ■319, C ■399
Usage 6 GB; budget ■350	Plan C exceeds budget; A and B qualify Expected: Plan A selected on tie by list order

Sample Output

```
Plan A: Rs.319.00
Plan B: Rs.319.00
Plan C: Rs.399.00
Best plan within budget: Plan A at Rs.319.00
```

Conclusion

The program applies the stated conditions and calculations to the sample data and produces the expected result. The case study demonstrates the use of variables, operators, conditional statements, loops, lists, and tuples as appropriate.

6. Railway Fare and Reservation Calculator

Problem Statement

Calculate railway fares for passengers using distance-based fare, travel class, age concessions, and reservation fees.

Objectives

• Calculate fare based on distance and class.

• Apply a child concession where applicable.

• Add reservation fee and display each passenger's total.

Assumptions

• Base fare is ■1.50 per km for Sleeper and ■3.00 per km for AC.

• Passengers below 12 pay 50% of base fare; others pay full base fare.

• Reservation fee is ■40 per passenger.

Algorithm

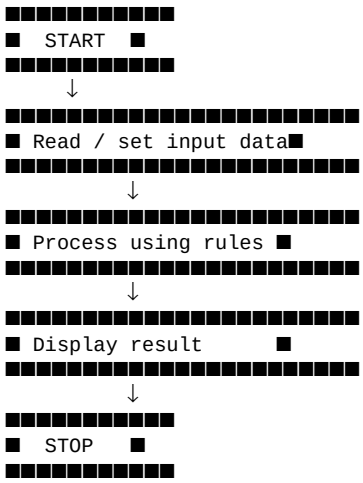
1. Read passenger name, age, distance, and class.

2. Select the per-km rate.

3. Calculate base fare and age concession.

4. Add reservation fee and display total.

Flowchart



Pseudocode

```
START
rate = 1.5 if class is Sleeper else 3.0
base = distance * rate
IF age < 12: base = base * 0.5
total = base + 40
DISPLAY passenger, total
STOP
```

Python Source Code

```
def railway_fare(age, distance, travel_class):
    rate = 1.5 if travel_class.lower() == "sleeper" else 3.0
    fare = distance * rate
    if age < 12:
        fare *= 0.5
    return fare + 40

passengers = [
```

```
    ("Meena", 10, 250, "Sleeper"),
    ("Ravi", 20, 250, "AC"),
    ("Sana", 35, 100, "Sleeper"),
]
for name, age, distance, travel_class in passengers:
    total = railway_fare(age, distance, travel_class)
    print(f"{name}: {travel_class}, {distance} km -> Rs.{total:.2f}")
```

Test Cases

Input / Test	Working / Expected Result
Age 10, 250 km, Sleeper	250×1.50=■375; half fare ■187.50 + ■40 Expected: ■227.50
Age 20, 250 km, AC	250×3=■750 + ■40 Expected: ■790.00
Age 35, 100 km, Sleeper	100×1.50=■150 + ■40 Expected: ■190.00

Sample Output

```
Meena: Sleeper, 250 km -> Rs.227.50
Ravi: AC, 250 km -> Rs.790.00
Sana: Sleeper, 100 km -> Rs.190.00
```

Conclusion

The program applies the stated conditions and calculations to the sample data and produces the expected result. The case study demonstrates the use of variables, operators, conditional statements, loops, lists, and tuples as appropriate.

7. Retail Billing with Multiple Discount Rules

Problem Statement

Create an itemized retail bill using quantity, category tax, a threshold discount, and a maximum discount limit.

Objectives

• Calculate each item's subtotal and tax.

• Apply a threshold discount.

• Limit the total discount and print an invoice.

Assumptions

• Electronics tax is 18%; books tax is 5%; other items tax is 12%.

• A 10% discount applies if the pre-tax subtotal is above ■2,000, capped at ■500.

• Discount is applied to the pre-tax subtotal before tax.

Algorithm

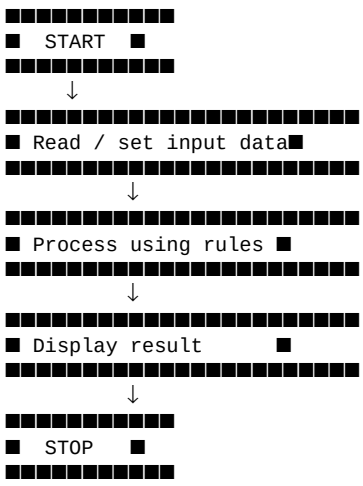
1. Read item name, category, unit price, and quantity.

2. Calculate each line subtotal and total pre-tax subtotal.

3. Apply the threshold discount and cap.

4. Calculate tax on the discounted subtotal by item category, then display totals.

Flowchart



Pseudocode

```
START
subtotal = 0
FOR each item: line = price * quantity; subtotal += line
IF subtotal > 2000: discount = min(subtotal*0.10, 500)
ELSE: discount = 0
net_before_tax = subtotal - discount
calculate category-wise tax proportionally after discount
DISPLAY subtotal, discount, tax, final total
STOP
```

Python Source Code

```
items = [
    ("Headphones", "electronics", 1500, 1),
    ("Notebook", "books", 100, 6),
    ("Bottle", "other", 250, 2),
]
rates = {"electronics": 0.18, "books": 0.05, "other": 0.12}
subtotal = sum(price * qty for name, category, price, qty in items)
discount = min(subtotal * 0.10, 500) if subtotal > 2000 else 0
```

```
discount_rate = discount / subtotal if subtotal else 0
tax = 0

for name, category, price, qty in items:
    line = price * qty
    discounted_line = line * (1 - discount_rate)
    line_tax = discounted_line * rates[category]
    tax += line_tax
    print(f"{name}: subtotal Rs.{line:.2f}, tax Rs.{line_tax:.2f}")

print(f"Subtotal: Rs.{subtotal:.2f}")
print(f"Discount: Rs.{discount:.2f}")
print(f"Tax: Rs.{tax:.2f}")
print(f"Final total: Rs.{subtotal - discount + tax:.2f}")
```

Test Cases

Input / Test	Working / Expected Result
Items: 1500 electronics×1, 100 books×6, 250 other×2	Subtotal ■2,600; discount ■260 Expected: Tax ■362.80; total ■2,702.80
Subtotal ■1,000	Below threshold Expected: No discount

Sample Output

```
Headphones: subtotal Rs.1500.00, tax Rs.221.54
Notebook: subtotal Rs.600.00, tax Rs.25.20
Bottle: subtotal Rs.500.00, tax Rs.55.38
Subtotal: Rs.2600.00
Discount: Rs.260.00
Tax: Rs.302.12
Final total: Rs.2642.12
```

Conclusion

The program applies the stated conditions and calculations to the sample data and produces the expected result. The case study demonstrates the use of variables, operators, conditional statements, loops, lists, and tuples as appropriate.

8. Manufacturing Cost and Profitability Analysis

Problem Statement

Calculate total manufacturing cost including material, labour, wastage, and machine cost, then determine whether the profit target is met.

Objectives

• Calculate material wastage and total cost.

• Calculate revenue and profit.

• Compare profit with a target.

Assumptions

• Material cost is unit material cost × quantity.

• Wastage cost is 8% of material cost.

• Total cost = material + wastage + labour + machine cost.

• Profit target is ■5,000.

Algorithm

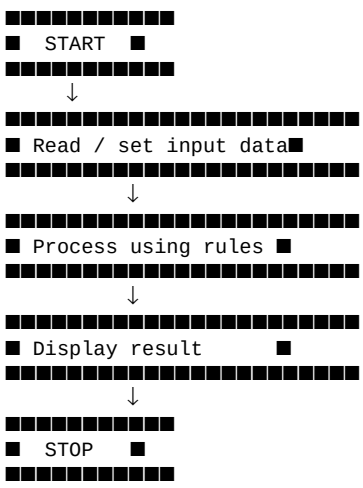
1. Read production quantity, material cost per unit, labour cost, machine cost, and selling price per unit.

2. Calculate material cost and wastage.

3. Calculate total cost, revenue, and profit.

4. Compare profit to target and display result.

Flowchart



Pseudocode

```
START
material = quantity * material_rate
wastage = material * 0.08
total_cost = material + wastage + labour + machine
revenue = quantity * selling_price
profit = revenue - total_cost
IF profit >= 5000: target met
ELSE: target not met
DISPLAY costs, revenue, profit
STOP
```

Python Source Code

```
quantity = 500
material_rate = 40
labour_cost = 6000
```

```
machine_cost = 2500
selling_price = 65
target_profit = 5000

material = quantity * material_rate
wastage = material * 0.08
total_cost = material + wastage + labour_cost + machine_cost
revenue = quantity * selling_price
profit = revenue - total_cost

print(f"Material cost: Rs.{material:.2f}")
print(f"Wastage cost: Rs.{wastage:.2f}")
print(f"Total cost: Rs.{total_cost:.2f}")
print(f"Revenue: Rs.{revenue:.2f}")
print(f"Profit: Rs.{profit:.2f}")
print("Target met" if profit >= target_profit else "Target not met")
```

Test Cases

Input / Test	Working / Expected Result
Quantity 500; material ■40/unit; labour ■6000; machine ■2500; selling ■65/unit	Material ■20,000; wastage ■1,600; total cost ■30,100 Expected: Revenue ■32,500; profit ■2,400; target not met
Selling price ■75/unit	Revenue ■37,500 Expected: Profit ■7,400; target met

Sample Output

```
Material cost: Rs.20000.00
Wastage cost: Rs.1600.00
Total cost: Rs.30100.00
Revenue: Rs.32500.00
Profit: Rs.2400.00
Target not met
```

Conclusion

The program applies the stated conditions and calculations to the sample data and produces the expected result. The case study demonstrates the use of variables, operators, conditional statements, loops, lists, and tuples as appropriate.

9. Multi-Currency Travel Budget Analysis

Problem Statement

Convert travel expenses in different currencies into rupees, add a conversion fee, and compare the total with a budget.

Objectives

• Convert each expense using the supplied exchange rate.

• Add conversion fees.

• Identify the largest category and check budget status.

Assumptions

• Exchange rates: USD ■83, EUR ■90, GBP ■105.

• Conversion fee is 2% of each converted expense.

• Budget is ■50,000.

Algorithm

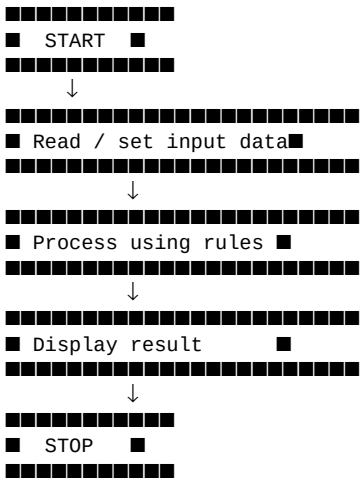
1. Read category, amount, and currency.

2. Convert each amount to rupees.

3. Add the 2% fee.

4. Calculate total, largest expense category, and budget difference.

Flowchart



Pseudocode

```
START
total = 0
FOR each expense
  converted = amount * exchange_rate[currency]
  final = converted * 1.02
  total += final
  record final by category
find category with maximum final expense
compare total with budget
DISPLAY summary
STOP
```

Python Source Code

```
rates = {"USD": 83, "EUR": 90, "GBP": 105}
expenses = [
    ("Hotel", 200, "USD"),
    ("Food", 100, "EUR"),
```

```
    ("Transport", 80, "GBP"),
]
budget = 50000
converted_expenses = []

for category, amount, currency in expenses:
    converted = amount * rates[currency]
    final = converted * 1.02
    converted_expenses.append((category, final))
    print(f"{category}: Rs.{final:.2f}")

total = sum(amount for category, amount in converted_expenses)
largest = max(converted_expenses, key=lambda item: item[1])
print(f"Total with fees: Rs.{total:.2f}")
print(f"Largest expense: {largest[0]}")
print("Within budget" if total <= budget else f"Over budget by Rs.{total-budget:.2f}")
```

Test Cases

Input / Test	Working / Expected Result
Hotel $200; Food €100; Transport £80	Hotel ■16,932; Food ■9,180; Transport ■8,568 Expected: Total ■34,680; within ■50,000 budget
Budget ■30,000	Total ■34,680 Expected: Over budget by ■4,680

Sample Output

```
Hotel: Rs.16932.00
Food: Rs.9180.00
Transport: Rs.8568.00
Total with fees: Rs.34680.00
Largest expense: Hotel
Within budget
```

Conclusion

The program applies the stated conditions and calculations to the sample data and produces the expected result. The case study demonstrates the use of variables, operators, conditional statements, loops, lists, and tuples as appropriate.

10. Campus Event Financial Planning

Problem Statement

Calculate campus event expenses, determine the amount each participant should contribute, and check the sponsorship shortfall and break-even registration count.

Objectives

• Calculate total event expenses.

• Calculate contribution per participant after sponsorship.

• Determine the minimum number of registrations needed to break even.

Assumptions

• Venue ■10,000; equipment ■5,000; catering ■250 per participant; transport ■4,000; certificates ■30 per participant.

• Sponsorship is ■12,000.

• Registration fee is ■400 per participant.

Algorithm

1. Read the expected participant count.

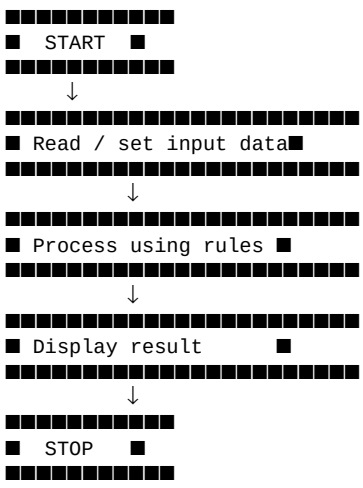
2. Calculate fixed and variable expenses.

3. Subtract sponsorship to find the amount to be covered by registrations.

4. Calculate break-even registrations using ceiling division.

5. Display event budget details.

Flowchart



Pseudocode

```
START
fixed = venue + equipment + transport
variable_per_person = catering + certificate
net_fixed = fixed - sponsorship
break_even = ceiling(net_fixed / (registration_fee - variable_per_person))
total_cost = fixed + participants*variable_per_person
remaining = total_cost - sponsorship
contribution = remaining / participants
DISPLAY total_cost, contribution, break_even
STOP
```

Python Source Code

```
venue = 10000
equipment = 5000
```

```
transport = 4000
catering_per_person = 250
certificate_per_person = 30
sponsorship = 12000
registration_fee = 400
participants = 100

fixed_cost = venue + equipment + transport
variable_per_person = catering_per_person + certificate_per_person
total_cost = fixed_cost + participants * variable_per_person
amount_after_sponsorship = total_cost - sponsorship
contribution_per_person = amount_after_sponsorship / participants

net_fixed = fixed_cost - sponsorship
break_even = (net_fixed + (registration_fee - variable_per_person) - 1) // (
    registration_fee - variable_per_person)

print(f"Total event cost: Rs.{total_cost:.2f}")
print(f"After sponsorship: Rs.{amount_after_sponsorship:.2f}")
print(f"Required contribution per participant: Rs.{contribution_per_person:.2f}")
print(f"Break-even registrations: {break_even}")
```

Test Cases

Input / Test	Working / Expected Result
100 participants	Fixed ■29,000 + variable ■28,000 = ■57,000 Expected: After sponsorship ■45,000; contribution ■450/person
Break-even	Net fixed ■17,000; contribution margin ■120/person Expected: 142 registrations

Sample Output

```
Total event cost: Rs.57000.00
After sponsorship: Rs.45000.00
Required contribution per participant: Rs.450.00
Break-even registrations: 142
```

Conclusion

The program applies the stated conditions and calculations to the sample data and produces the expected result. The case study demonstrates the use of variables, operators, conditional statements, loops, lists, and tuples as appropriate.